Page 1

BookMyShow System Design

@abhi_techhub

☆ Complete HLD + LLD Handbook ☆

CASE STUDY
Book Movies.
Book Experiences.
Scale Worldwide.

1. Functional Requirements

- User Registration / Login
- Search Movies
- Search Theatres (by city, location)
- View Movies Details
- View Theatres Details
- View Show Timings
- Select Show
- Select Seats
- Book Tickets
- Make Payment
- Booking Confirmation
- Ticket (QR / Barcode) Generation
- Cancellation
- Refund
- Notifications (Email / SMS / Push)
- Offers & Coupons

2. Non-Functional Requirements

- High Availability (99.9%+)
- Low Latency (Fast Search & Booking)
- Scalability (Millions of users)
- Fault Tolerance
- Data Consistency
- Security (PCI DSS for payments)
- Reliability
- Elasticity
- Monitoring & Observability
- Maintainability
- Cost Efficiency

3. Functional vs Non-Functional

Functional	Non-Functional
Define what the system should do.	Define how well the system should do it.
User visible features.	Quality attributes of the system.
Examples: Search, Book, Payment, Cancel	Examples: Scalability, Security, Availability
Change frequently as per business.	Change rarely, stable over time.

4. Core Entities (Key Building Blocks)

User
Person who uses the platform to search, book & manage tickets.
Key: user_id

Movie
Contains movie details like name, genre, language, duration, etc.
Key: movie_id

Theatre
Physical location where movies are screened.
Key: theatre_id

Screen
A theatre can have multiple screens.
Key: screen_id

Show
A particular movie playing at a screen at a specific time.
Key: show_id

Seat
Indivicual seat in a screen with a unique number.
Key: seat_id

Booking
Represents the ticket booked by a user for a show.
Key: booking_id

Payment
Payment details for a booking transaction.
Key: payment_id

5. Assumptions & Constraints

- Each user has a unique account.
- A movie can have multiple shows in different theatres.
- A show belongs to one screen, and one screen belongs to one theatre.
- Seats in a show must be unique.
- Once a seat is booked, it cannot be booked by another user.
- Seat hold/lock is temporary (e.g., 5-10 minutes).
- Payments are processed through third-party gateways.
- Refunds follow business rules (e.g., time before show).
- System should handle peak traffic during releases.
- All times are stored in UTC.

High Level Flow (At a Glance)

User → Search Movies / Theatres → Select Show → Select Seats → Make Payment → Booking Confirmed

Goal

Design a scalable, reliable and high performance system that allows users to discover movies, book tickets seamlessly and ensures a smooth experience even during massive traffic.

Interview Tip

Always start by understanding requirements properly

Pro Tip

✓ Focus on the core flow first

Remember

☆ User Experience is key.

BookMyShow System Design

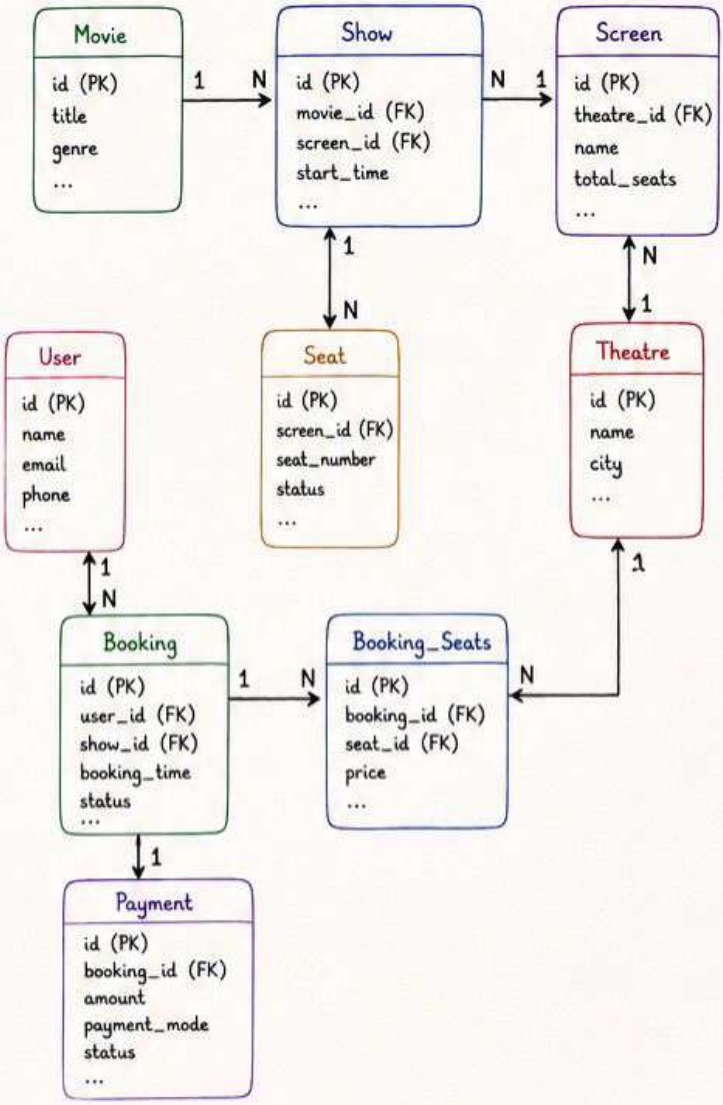
@abhi_techhub

☆ Complete HLD + LLD Handbook ☆

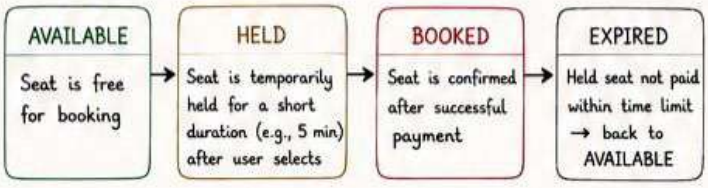
1. Database Design (Core Tables)

Table Name	Important Fields	Description
users	id (PK), name, email, phone, password_hash, created_at	Stores user details and authentication info.
movies	id (PK), title, genre, language, duration, release_date, poster_url	Movie master data.
theatres	id (PK), name, city, address, lat, lng	Theatre information.
screens	id (PK), theatre_id (FK), name, total_seats	Screens inside a theatre.
shows	id (PK), movie_id (FK), screen_id (FK), start_time, end_time, price	A movie show at a particular time.
seats	id (PK), screen_id (FK), seat_number, seat_type, status	All seats in a screen (AVAILABLE / HELD / BOOKED)
bookings	id (PK), user_id (FK), show_id (FK), booking_time, total_amount, status	Booking record for a show.
booking_seats	id (PK), booking_id (FK), seat_id (FK), price	Mapping of seats for a booking.
payments	id (PK), booking_id (FK), amount, payment_mode, transaction_id, status	Payment transactions.

2. ER Diagram (Simplified)



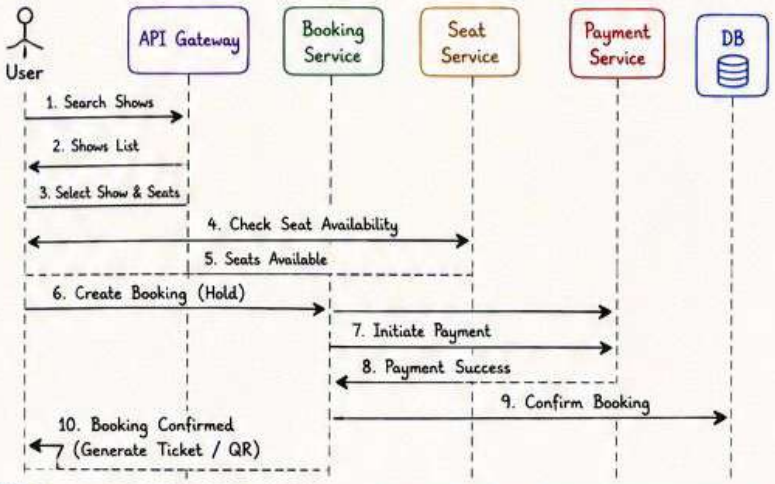
3. Seat Status Flow



4. Key Business Rules

- ✓ A seat can be booked by only one user for a show.
- ✓ Held seats will be released if payment is not completed in time.
- ✓ Users can cancel booking as per cancellation policy.
- ✓ Payments are processed securely through third-party gateways.
- ✓ Show timings and seat availability are real-time.
- ✓ Ticket (QR) is generated only after successful payment.

5. Sequence Diagram (Book Ticket - High Level)



6. Caching Strategy

Use Redis for:

- Popular Shows, Movies, Theatres
- Seat maps for active shows
- Show timings & Pricing

☆ Interview Tip

Always clarify:

1. What is the expected scale?

Pro Tip

Break the problem

Remember

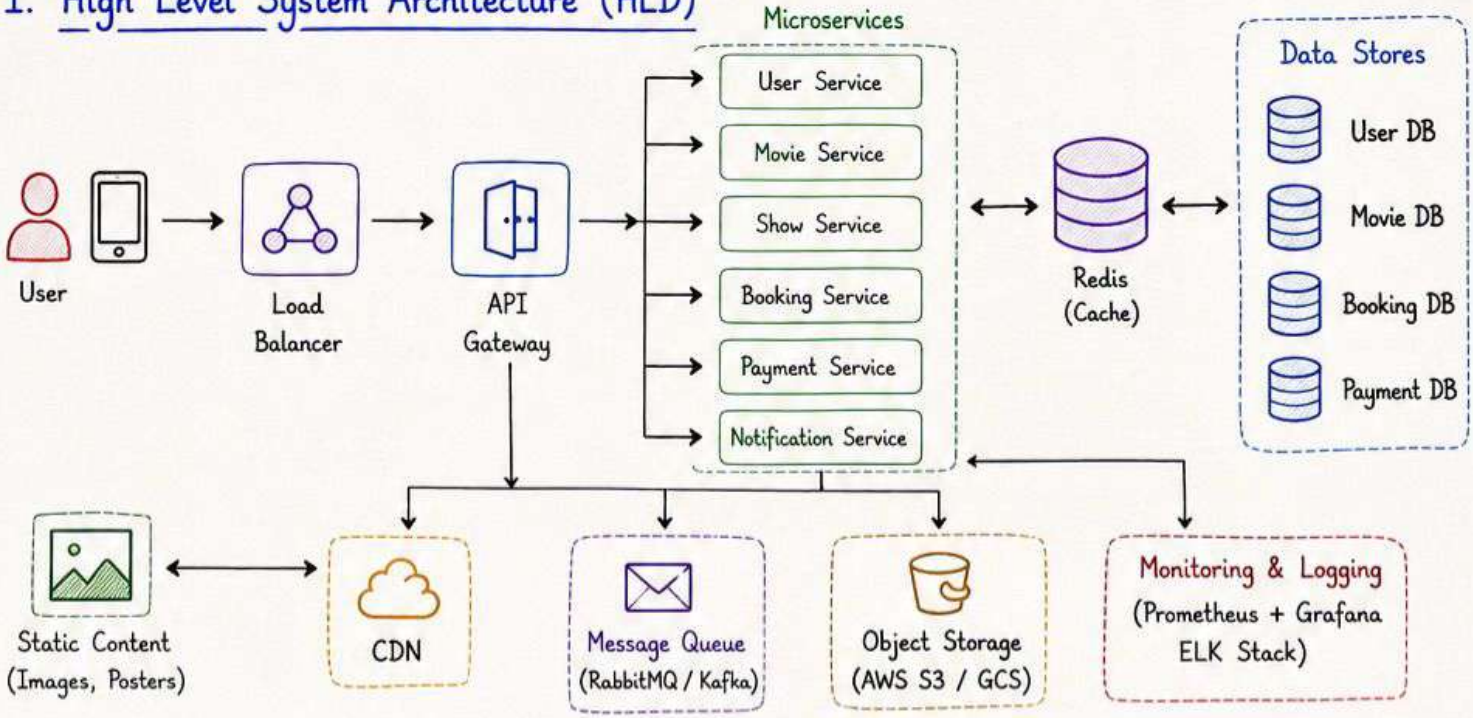
☆ User experience first.

BookMyShow System Design

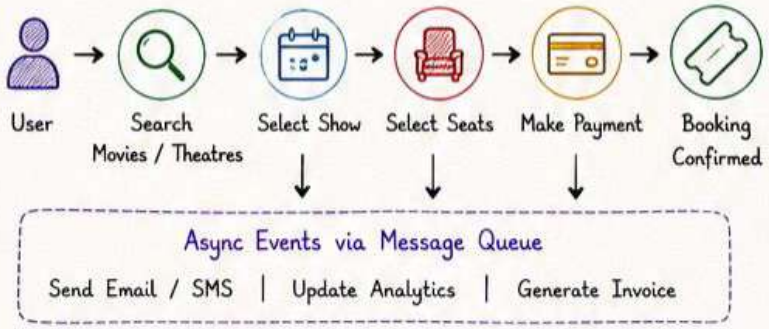
@abhi_techhub

☆ Complete HLD + LLD Handbook ☆

1. High Level System Architecture (HLD)



2. Data Flow (Happy Path)



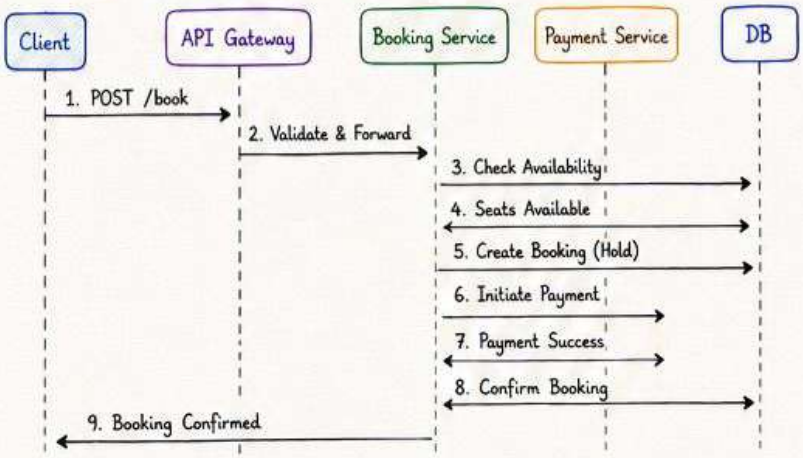
3. Key Components

API Gateway	Single entry for all client requests, routing & auth.
Microservices	Independent services for each domain.
Database	Persistent storage for each service.
Cache	High speed data access (sessions, shows, seats, etc.).
Message Queue	Decouple services, handle async tasks.
CDN	Fast delivery of static content.
Object Storage	Store large files (images, invoices, etc.).
Monitoring	Track system health, logs & alerts.

4. Microservices Breakdown

Service	Responsibility
User Service	Manage users, profiles, authentication, preferences.
Movie Service	Manage movies, genres, cast, language, trailers, etc.
Show Service	Manage theatres, screens, shows, timings, pricing.
Booking Service	Handle seat availability, holds, bookings.
Payment Service	Handle payments, refunds, transactions.
Notification Service	Send emails, SMS, push notifications.

5. API Flow Example (Book Ticket)



6. Scaling Strategies

- Horizontal scaling of microservices.
- Read replicas for databases.
- Caching for shows, movies, seat maps.

☆ Interview Tip

Always mention:

1. Scalability
2. Availability

Pro Tip

✓ Keep booking flow atomic

Remember

☆ Seat availability is real-time.

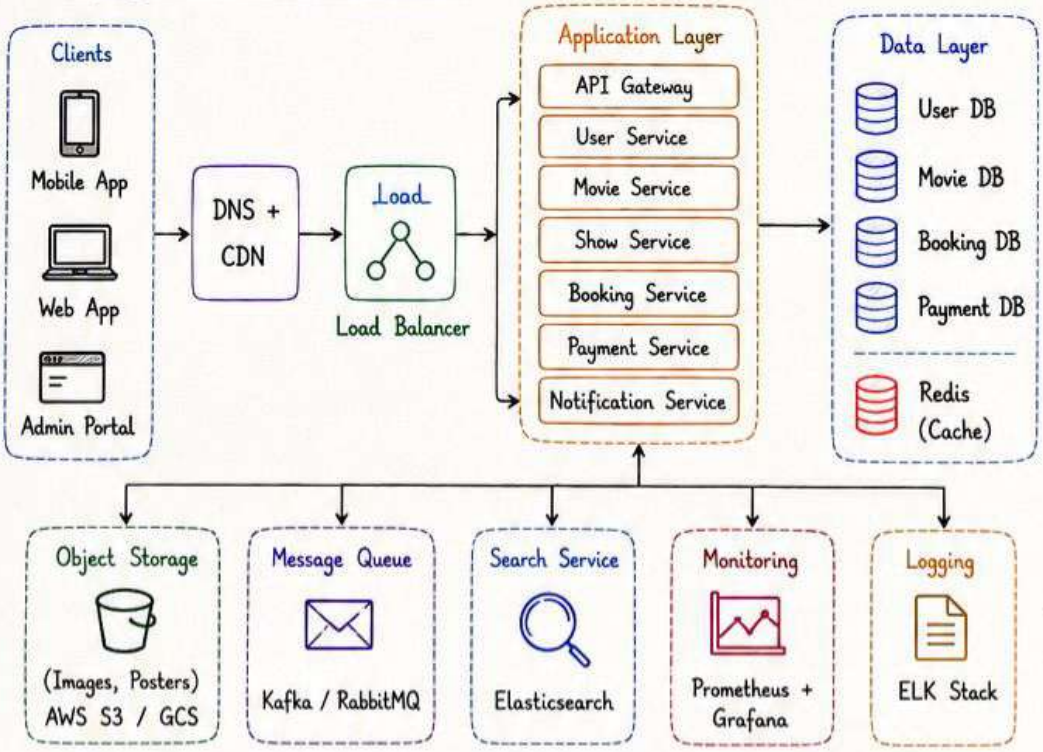
BookMyShow System Design

@abhi_techhub

Complete HLD + LLD Handbook

CASE STUDY
- Book Movie.
- Book Experience.
- Scale Worldwide.

1. Deployment Architecture (HLD)



2. Caching Strategy

- ✓ Cache frequently read data (Movies, Shows, Theatres, Seat maps)
- ✓ Use Redis for low latency access.
- ✓ Cache with TTL to keep data fresh.
- ✓ Invalidate / update cache on data changes.

What to Cache?
- Movie Details
- Show Timings
- Seat Availability (per show)
- Theatre & Screen Info
- Popular Searches / Filters
- Offers & Banners

3. Database Indexing Strategy

Table	Index Fields	Purpose
movies	(release_date), (genre)	Filter, sort, search movies
shows	(movie_id, show_date, show_time)	Quick show lookup
seats	(show_id, seat_number)	Check seat availability
bookings	(user_id), (show_id, booking_time)	User bookings & history
payments	(booking_id), (payment_mode)	Transaction tracking

☆ Use Composite Indexes where filtering on multiple columns is common.

4. Partitioning Strategy

- Partition shows table by show_date (monthly).
- Partition bookings table by booking_time (monthly).
- Helps in handling large data and improves query performance.

Example: bookings_2025_05
bookings_2025_06 ...

5. Scaling Strategy

- ✓ Horizontal scaling of microservices using containers (Docker + K8s).
- ✓ Read Replicas for databases.
- ✓ Cache to reduce DB load.
- ✓ CDN for static content (Posters, Images, JS, CSS).
- ✓ Auto-scaling based on traffic.
- ✓ Async processing for heavy tasks (Email, Notifications).

6. Failure Handling & Reliability

- Retry mechanism with exponential backoff.
- Circuit Breaker pattern for external services.
- Fallback responses for non-critical services.
- Idempotent APIs for booking & payment.
- Data backups & point-in-time recovery.
- Queue based processing for resilience.
- Health checks for all services.

7. Security Considerations

- HTTPS everywhere.
- JWT based authentication.
- Role based access control (RBAC).
- Rate limiting & throttling.
- PCI DSS compliant payments.
- Input validation & sanitization.
- Secure storage of sensitive data.

8. Monitoring & Observability

- Monitor service health (Prometheus + Grafana)
- Centralized logging (ELK Stack)
- Alerts for errors, latency, high traffic, failures.
- Distributed tracing (Jaeger / Zipkin)

Interview Tip

Pro Tip

Remember

Page 5

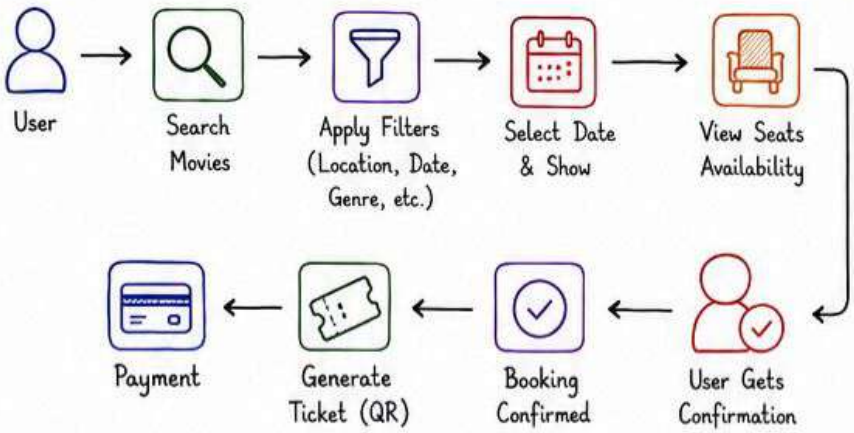
BookMyShow System Design

Complete HLD + LLD Handbook

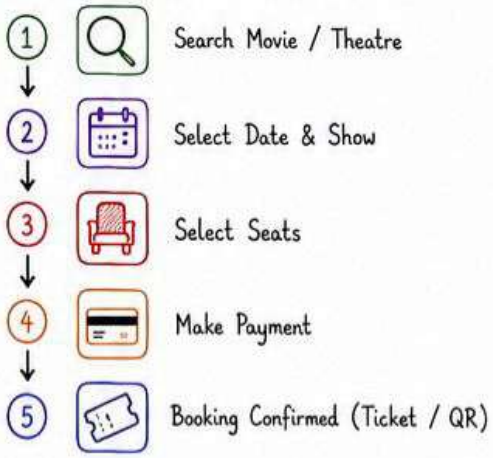
@abhi_techhub

CASE STUDY
- Book Movie.
- Book Experience.
- Scale Worldwide.

1. Movie Search Flow (High Level)



2. Show Booking Process (Detailed Flow)



3. Data Model (Key Entities)

User
- user_id (PK)
- name
- email
- phone
- password_hash
- created_at
- ...

Movie
- movie_id (PK)
- title
- genre
- duration
- language
- release_date
- ...

Theatre
- theatre_id (PK)
- name
- location_id (FK)
- address
- ...

Screen
- screen_id (PK)
- theatre_id (FK)
- name
- total_seats
- ...

Show
- show_id (PK)
- movie_id (FK)
- screen_id (FK)
- show_date
- start_time
- ...

Seat
- seat_id (PK)
- screen_id (FK)
- seat_number
- seat_type
- ...

Booking
- booking_id (PK)
- user_id (FK)
- show_id (FK)
- booking_time
- status
- ...

Booking_Seat
- booking_id (FK)
- seat_id (FK)
- price
- ...

(PK: booking_id, seat_id)

Payment
- payment_id (PK)
- booking_id (FK)
- amount
- payment_mode
- status
- ...

Location
- location_id (PK)
- city
- state
- country
- ...

4. Key Design Decisions

- Microservices architecture for scalability and independent deployments.
- Separate read and write DBs for high availability.
- Use Redis for caching shows, theatres, seat maps and user sessions.
- Event driven communication using Kafka (Booking, Payment, Notification).
- Idempotent APIs for booking & payment to handle retries.
- CQRS pattern for heavy read traffic (search & listings).

5. API Endpoints (Examples)

Method	Endpoint	Description
GET	/v1/movies	List movies with filters
GET	/v1/shows	List shows for a movie on a date
GET	/v1/shows/{show_id}/seats	Get seat layout & availability
POST	/v1/bookings	Create a booking (hold seats)
POST	/v1/payments	Make payment for a booking
GET	/v1/bookings/{id}	Get booking details
GET	/v1/users/{id}/bookings	List user bookings

6. Non-Functional Requirements

- High Availability (99.95%+)
- Low Latency (< 200ms for search)
- Scalability (Millions of users)
- Secure (PCI DSS, HTTPS, RBAC)
- Data Consistency (Bookings)
- Monitoring & Alerting
- Disaster Recovery & Backup

7. Technology Stack (Example)

Frontend	Web: React / Next.js Mobile: Flutter
Backend	Java / Spring Boot (Microservices)
Database	MySQL (Primary), Read Replicas
Cache	Redis
Message Queue	Kafka
Search	Elasticsearch
Storage	AWS S3 (Images, Posters)
Monitoring	Prometheus + Grafana
Logging	ELK Stack

8. Bottlenecks & Mitigation

Bottleneck	Mitigation
High read traffic on search	Caching (Redis), Elasticsearch
Seat locking & concurrency	Distributed locks, Idempotency

9. Metrics to Track

- Search latency (p95 / p99)
- Booking success rate
- Payment success rate

Interview Tip

- Start with high level design.
- Clarify requirements & assumptions.
- Divide into components & flows.

Page 6

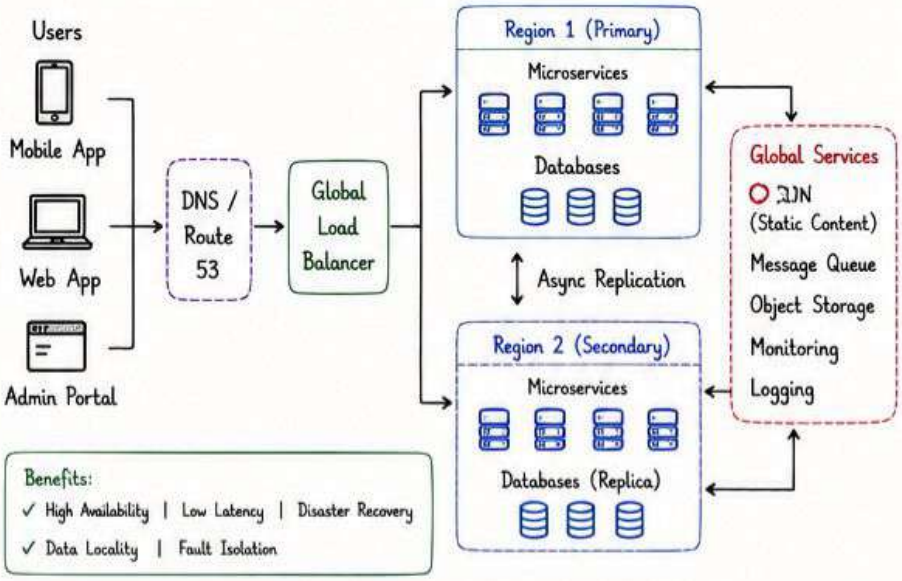
BookMyShow System Design

@abhi_techhub

Complete HLD + LLD Handbook

CASE STUDY
- Book Movie.
- Book Experience.
- Scale Worldwide.

1. Global Architecture (Multi-Region)

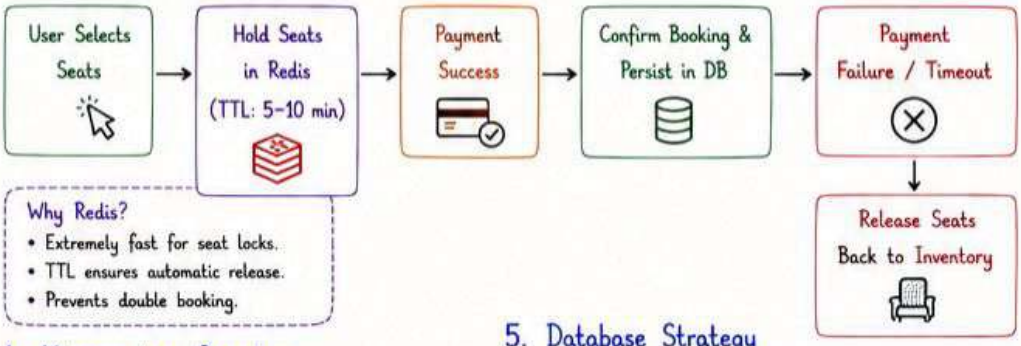


Benefits:
✓ High Availability | Low Latency | Disaster Recovery
✓ Data Locality | Fault Isolation

2. Ticket Booking Flow (Detailed)

1. Search Movies / Theatres
2. Select Date & Show
3. Select Seats
4. Hold Seats (5-10 mins)
5. Make Payment
6. Confirm Booking
7. Generate Ticket (QR)
8. Send Ticket (Email / SMS / In-App)

3. Seat Selection - Hold Mechanism



Why Redis?
- Extremely fast for seat locks.
- TTL ensures automatic release.
- Prevents double booking.

4. Microservices Overview

Service	Responsibility
User Service	User management, authentication, profiles.
Movie Service	Movies, genres, ratings, language, trailers.
Theatre Service	Theatre information, screens, seat maps.
Show Service	Shows, timings, pricing, availability.
Booking Service	Seat holds, bookings, cancellations.
Payment Service	Payment processing, refunds, transactions.
Notification Service	Email, SMS, Push notifications.
Search Service	Search & suggestions (Elasticsearch).

5. Database Strategy

Database	Type	Used For
User DB	MySQL (Primary)	User data, auth, profiles
Movie DB	MySQL (Primary)	Movies, genres, ratings
Booking DB	MySQL (Primary)	Bookings, payments, txn
Theatre DB	MySQL (Primary)	Theatres, screens, seats
Search Index	Elasticsearch	Search, filters, suggestions
Cache	Redis	Session, holds, counters
Analytics DB	ClickHouse / Druid	Reports, dashboards
Object Storage	S3 / GCS	Posters, banners, invoices

Use Read Replicas for heavy read queries.
Sharding for bookings & users based on user_id / region.

6. Message Queue Usage

- **Booking Created** — Send Email, SMS, Update Analytics
- **Payment Success** — Generate Invoice, Update Ledger
- **Show Reminder** — Send reminder 1 hour before show
- **Analytics Events** — User actions, search, conversions

Tech: Kafka / RabbitMQ

7. API Design Principles

- ✓ RESTful APIs with resource-based URIs.
- ✓ Stateless, secure & versioned APIs.
- ✓ Pagination, filtering & sorting.
- ✓ Rate limiting & throttling.
- ✓ Idempotent APIs for critical operations.
- ✓ Consistent error handling & responses.

Standard Response Format

```
{
  "success": true,
  "code": 200,
  "message": "Booking confirmed",
  "data": { ... }
}
```

8. Security Considerations

- JWT based authentication
- RBAC for admin & partners
- PCI DSS compliant payments
- WAF, DDoS protection
- Input validation & sanitization
- Encrypt sensitive data (at rest & in transit)

9. Monitoring & Observability

- Metrics: Prometheus + Grafana
- Logs: ELK Stack (Elasticsearch, Logstash, Kibana)
- Alerts: PagerDuty / Opsgenie
- Tracing: Jaeger / Zipkin
- Uptime & SLA Monitoring

Page 7

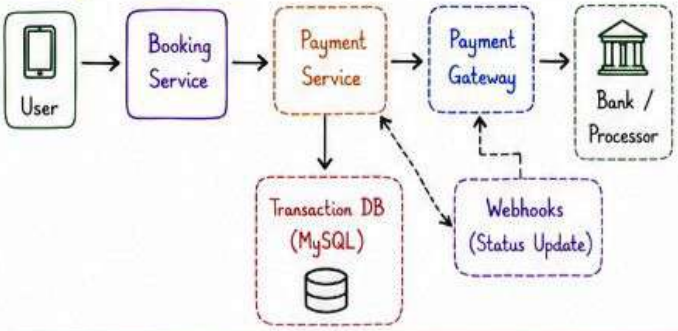
BookMyShow System Design

@abhi_techhub

Complete HLD + LLD Handbook

CASE STUDY
- Book Movie.
- Book Experience.
- Scale Worldwide.

1. Payment System Architecture



Payment Flow:
1. User initiates payment.
2. Payment Service creates order & calls Gateway.
3. User redirected to Gateway (UPI / Card / Wallet / NetBanking).
4. Gateway processes & sends webhook.
5. Payment Service updates status in DB.
6. Booking confirmed on success.

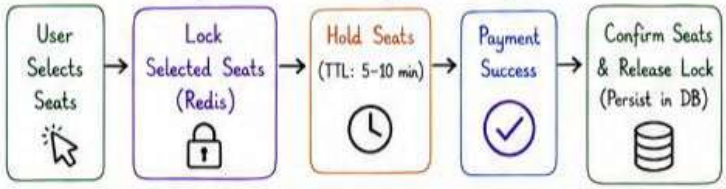
Features:
- ✓ Idempotent payment APIs
- ✓ Retry on failure
- ✓ Webhook verification
- ✓ 3DS for card payments
- ✓ Refund & reconciliation

2. Database Design - Key Tables

Table Name	Important Fields	Description
users	id (PK), name, email, phone, city	Stores user info
movies	id (PK), title, genre_id, duration, language, release_date	Movie master data
theatres	id (PK), name, city_id, address	Theatre information
screens	id (PK), theatre_id (FK), name, total_seats	Screens in a theatre
shows	id (PK), movie_id (FK), screen_id (FK), show_date, start_time, price	Movie shows
seats	id (PK), screen_id (FK), seat_number, seat_type, status	Seat layout & status
bookings	id (PK), user_id (FK), show_id (FK), booking_time, total_amount, status	Booking header
booking_seats	id (PK), booking_id (FK), seat_id (FK), price	Seats in a booking
payments	id (PK), booking_id (FK), amount, payment_mode, txn_id, status	Payment transactions

☆ Use proper Indexes on FK columns for performance.

3. Seat Booking - Concurrency Handling



Why Redis Lock?
- Prevents double booking of same seats.
- Fast in-memory locking with TTL.
- Auto release on timeout or failure.

Lock Key Example:

lock:showId:seatId → value: userId

TTL: 5-10 minutes

4. Caching Strategy

Cache What?
- Popular Movies / Shows
- Theatre & Screen Info
- Seat Maps (Static)
- Show Timings
- Offers & Banners

How?
- Redis for fast access
- TTL based expiration
- Cache warming for trending content
- Invalidate on data changes

Benefit: Reduced DB load, Faster response, Better UX

5. Search & Recommendation

Search (Elasticsearch)
- Movies, Theatres, Events
- Auto-complete
- Filters: City, Date, Language, Genre
- Faceted Search

Recommendations
- Based on user behavior
- Collaborative filtering
- Trending content
- Personalized suggestions

6. CDN & Static Content

CDN (CloudFront / Akamai)
- Posters, Banners, Images
- JS, CSS, Fonts
- Static Pages

→ Faster global delivery

Store in Object Storage

AWS S3 / GCS
- Images, Videos
- Documents
- Backups

7. Scalability Strategies

- ✓ Horizontal scaling of microservices
- ✓ Auto-scaling groups on traffic
- ✓ Read Replicas for databases
- ✓ Sharding for bookings by date/region
- ✓ CDN for static & dynamic content
- ✓ Caching to reduce DB load
- ✓ Async processing for heavy tasks

8. Monitoring & Observability

- Metrics: Prometheus + Grafana
- Logs: ELK Stack (Elasticsearch, Logstash, Kibana)
- Alerts: PagerDuty / Opsgenie
- Tracing: Jaeger / Zipkin
- Uptime & SLA Monitoring

9. High Availability & Disaster Recovery

High Availability
- Multi-AZ deployment
- Load balancer health checks
- Replicated databases
- Stateless services

Disaster Recovery
- Cross-region backups
- Point-in-time recovery
- Data replication
- Regular DR drills

Key Metrics to Track
- Booking Success Rate (%)
- Payment Success Rate (%)
- Seat Hold Failure Rate (%)
- Average Response Time (ms)
- Active Users (DAU / MAU)
- System Uptime (%)

Page 8

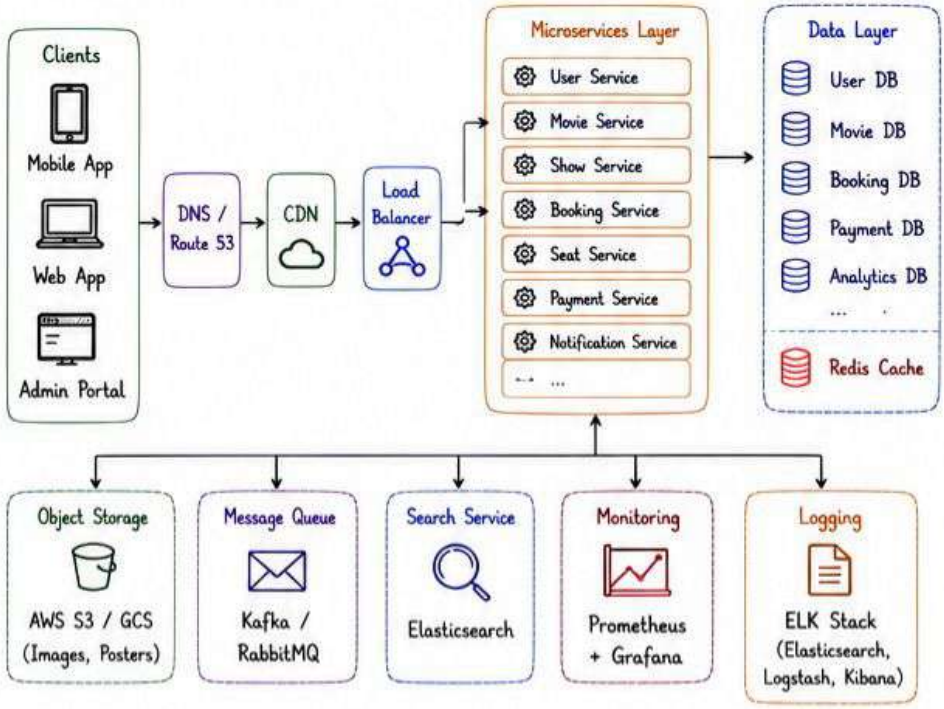
BookMyShow System Design

@abhi_techhub

Complete HLD + LLD Handbook

CASE STUDY
- Book Movie.
- Book Experience.
- Scale Worldwide.

1. Complete System Architecture (End-to-End)



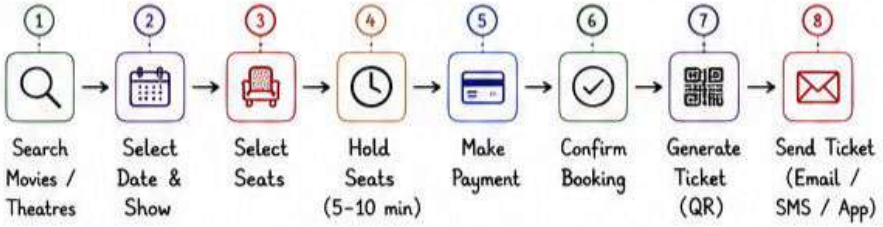
2. API Gateway Responsibilities

- ✓ Authentication & Authorization
- ✓ Rate Limiting & Throttling
- ✓ Request Routing to Services
- ✓ Request/Response Transformation
- ✓ Caching for GET requests
- ✓ Monitoring & Logging
- ✓ Error Handling & Retries

3. Core Microservices

Service	Responsibility
User Service	User management, profiles, auth
Movie Service	Movies, genres, content, ratings
Show Service	Shows, timings, pricing
Seat Service	Seat layout, availability, locks
Booking Service	Bookings, cancellations, hold
Payment Service	Payments, refunds, transactions
Notification Service	Email, SMS, Push notifications
...	...

4. Booking Flow (End-to-End)



Seat hold uses Redis with TTL (5-10 min). If payment fails, seats are auto-released.

5. Data Consistency & Transactions

- ✓ Use DB transactions for booking creation & payment.
- ✓ Idempotent APIs to handle retries safely.
- ✓ Unique constraints on booking_id to prevent duplicates.
- ✓ Outbox pattern for reliable notifications.
- ✓ Saga pattern for distributed transactions.

6. Database Partitioning & Indexing

Partitioning Strategy

Table	Partition By
shows	show_date (Monthly)
bookings	booking_time (Monthly)
payments	payment_time (Monthly)
analytics_events	event_date (Daily)

Important Indexes

Table	Index Fields
shows	(movie_id, show_date, show_time)
seats	(show_id, seat_number)
bookings	(user_id, show_id, booking_time)
payments	(booking_id, payment_mode)

Use Composite Indexes where filtering on multiple columns is common.

7. Caching Strategy

Cache What?
- Movie Details
- Theatre & Screen Info
- Seat Maps (Static)
- Show Timings
- Popular Searches
- Offers & Banners

How?
- Redis for fast access
- TTL for expiry
- Cache warming for trending data
- Invalidate on updates or changes

Benefit: Reduced DB load, Faster response, Better UX

8. High Availability & Disaster Recovery

High Availability
- Multi-AZ deployment
- Auto-scaling groups
- Load balancer + Health checks
- Stateless microservices
- Replicated databases

Disaster Recovery
- Cross-region backups
- Point-in-time recovery
- Data replication
- Regular DR drills

9. Monitoring & Observability

- Metrics: Prometheus + Grafana
- Logs: ELK Stack (Elasticsearch, Logstash, Kibana)
- Alerts: PagerDuty / Opsgenie
- Tracing: Jaeger / Zipkin
- Uptime & SLA Monitoring

Key Metrics to Track
- Booking Success Rate (%)
- Payment Success Rate (%)
- Seat Hold Failure Rate (%)
- Average Response Time (ms)
- Active Users (DAU / MAU)
- System Uptime (%)

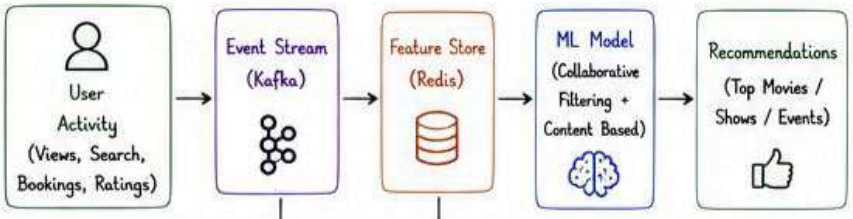
BookMyShow System Design

@abhi_techhub

Complete HLD + LLD Handbook

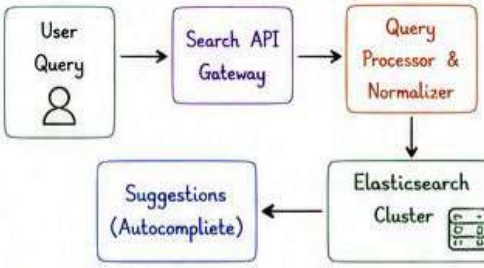
CASE STUDY
- Book Movie.
- Book Experience.
- Scale Worldwide.

1. Recommendation System



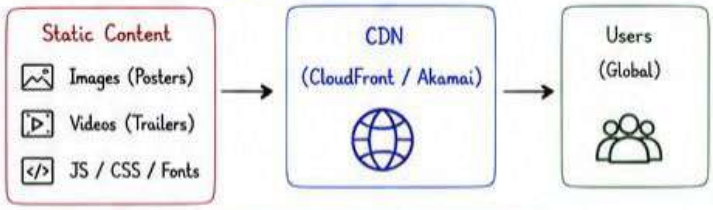
What we recommend?
- ✓ Personalized Home Page
- ✓ Similar Movies / You May Also Like
- ✓ Trending in Your City
- ✓ Because you watched / booked ...
- ✓ Continue Watching / Remind Me

2. Search Architecture



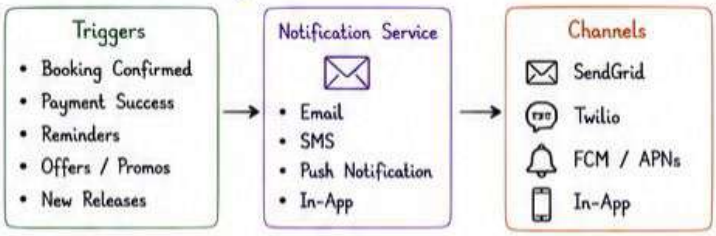
Search In:
- Movies, Shows, Events
- Theatres, Cities, Experiences
- People (Artists, Venues)

3. Content Delivery Network (CDN)



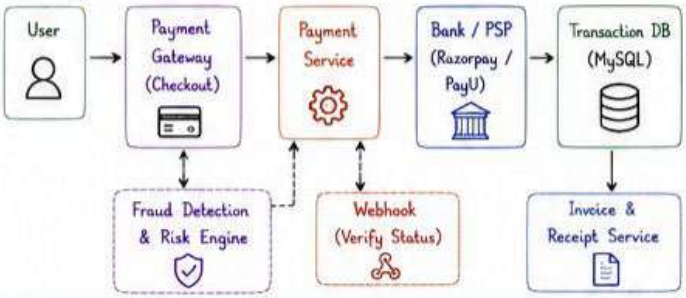
- Reduces latency, improves load time.
- Static assets cached at edge locations.

4. Notification System



Async processing using Kafka + Workers for reliability.

5. Payment Architecture



Key Points
- ✓ PCI DSS compliant
- ✓ Tokenization of cards
- ✓ Idempotent transactions
- ✓ Retry mechanism
- ✓ Refunds & Cancellations
- ✓ Reconciliation & Settlement

6. Scalability Patterns

Horizontal Scaling	Scale microservices horizontally using containers (Docker + K8s).
Auto Scaling	Scale based on CPU, memory or custom metrics.
Load Balancing	Distribute traffic across multiple instances.
Multi-Region Deployment	Active-active or active-passive for high availability.
Sharding	Shard databases by city/ region / user_id.

7. Security Best Practices

- HTTPS everywhrere
- JWT based authentication
- Role Based Access Control (RBAC)
- Rate limiting & throttling
- Input validation & sanitization
- PCI DSS compliance
- Secrets management (Vault)
- Audit logs & monitoring
- Encrypt data in transit and at rest

8. Monitoring & Observability

- Metrics: Prometheus + Grafana
- Logs: ELK Stack (Elasticsearch, Logstash, Kibana)
- Alerts: PagerDuty / Opsgenie
- Tracing: Jaeger / Zipkin
- Dashboards: Business + Technical KPIs

Important KPIs
- Booking Success Rate (%)
- Payment Success Rate (%)
- Seat Hold Failure Rate (%)
- Avg Response Time (ms)
- Active Users (DAU / MAU)
- System Uptime (%)

9. LLD Components (Examples)

Component	Responsibility	Key Classes / Tables
User Service	User CRUD, profiles, auth	User, Address, AuthToken
Movie Service	Movies, genres, ratings	Movie, Genre, Rating

10. Key Takeaways

- ✓ Design for scale, reliability & performance.
- ✓ Use microservices with separation of concerns.
- ✓ Cache smartly, index properly.
- ✓ Monitor everything.

Page 10

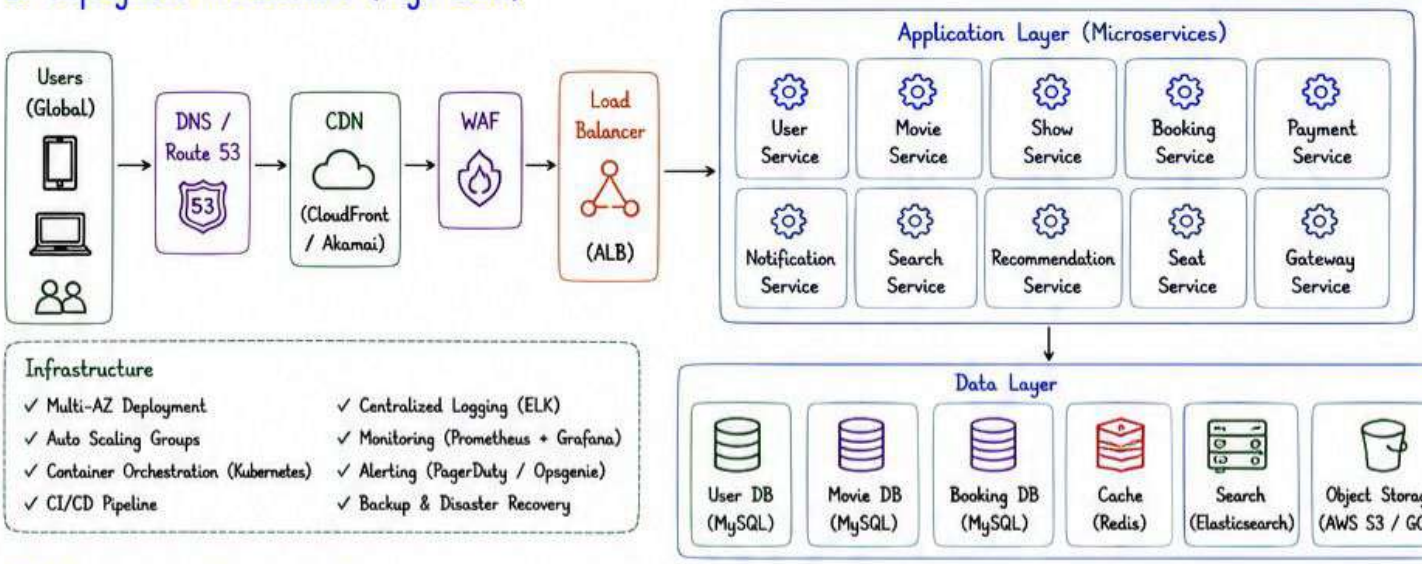
BookMyShow System Design

Complete HLD + LLD Handbook

@abhi_techhub

CASE STUDY
- Book Movie.
- Book Experience.
- Scale Worldwide.

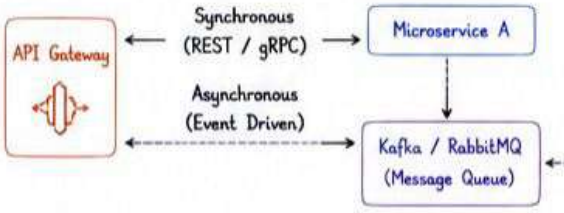
1. Deployment Architecture (High Level)



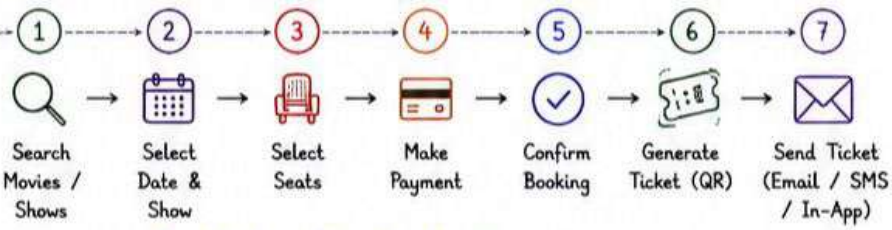
Infrastructure

- ✓ Multi-AZ Deployment
- ✓ Auto Scaling Groups
- ✓ Container Orchestration (Kubernetes)
- ✓ CI/CD Pipeline
- ✓ Centralized Logging (ELK)
- ✓ Monitoring (Prometheus + Grafana)
- ✓ Alerting (PagerDuty / Opsgenie)
- ✓ Backup & Disaster Recovery

2. Microservices Communication



3. Data Flow for Booking



4. Caching Layers

Layer	What is Cached?	Technology	TTL
Browser Cache	Static assets (JS, CSS, Images)	Browser Cache	Days
CDN Cache	Images, Posters, Trailers	CloudFront / Akamai	Hours - Days
Application Cache	Popular Movies, Shows, Seat Maps	Redis	Minutes
Database Cache	Query Results, Metadata	Redis / Memcached	Minutes
Object Cache	Session, Tokens	Redis	Minutes

5. Database Sharding Strategy

Strategy	Sharding Key	Use Case	Example
User Based	user_id	User data, bookings	user_id % N
Movie Based	movie_id	Shows, Reviews	movie_id % N
City Based	city_id	Theatres, Shows	city_id % N
Time Based	Date (Monthly)	Bookings, Payments	YYYY_MM
Hash Based	Hash (id)	Even data distribution	CRC32(id) % N

☆ Choose sharding key based on query pattern & data distribution.

6. Fault Tolerance & Resilience

- ✓ Circuit Breaker (Hystrix / Resilience4j)
- ✓ Retry with Exponential Backoff
- ✓ Timeouts for all external calls
- ✓ Bulkheads to isolate failure
- ✓ Graceful Degradation
- ✓ Health Checks & Heartbeats
- ✓ Idempotent APIs for safe retries

7. Observability Stack

- Metrics: Prometheus + Grafana
- Logs: ELK Stack (Elasticsearch, Logstash, Kibana)
- Tracing: Jaeger / Zipkin
- Alerts: PagerDuty / Opsgenie
- Dashboards: Business + Technical KPIs

8. Key KPIs to Monitor

- Booking Success Rate (%)
- Payment Success Rate (%)
- Seat Hold Failure Rate (%)
- Average Response Time (ms)
- Active Users (DAU / MAU)
- System Uptime (%)
- Revenue per Day
- Search Latency (p95 / p99)

9. Security Considerations

- HTTPS everywhere
- JWT based authentication
- Role Based Access Control (RBAC)
- PCI DSS compliant payments
- Secrets management (Vault / KMS)
- Input validation & sanitization
- Rate limiting & DDoS protection
- Audit logs & monitoring
- Encrypt data in transit & at rest

10. Non-Functional Requirements

- High Availability: 99.95%+
- Low Latency: < 200ms for search
- Scalability: Millions of users
- Security: PCI DSS, HTTPS, RBAC
- Data Consistency: Strong for bookings
- Disaster Recovery: RTO < 1 hr, RPO < 15 min
- Maintainability: Modular & Testable

System Design Checklist

- ✔ Requirements clearly understood
- ✔ HLD architecture designed
- ✔ Components & data flows identified
- ✔ Database schema & scaling planned
- ✔ APIs & contracts defined
- ✔ Caching, Queues & CDN planned
- ✔ Security & monitoring in place
- ✔ Failure handling & recovery planned
- ✔ Metrics & KPIs defined